Reviewer Pack — Coxeter Group Action Complexity of Boolean Functions

Entry 1cd48f6fed7a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 01:33:16 UTC

Coxeter Group Action Complexity of Boolean Functions

Entry ID: `1cd48f6fed7a` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-27 01:33:16 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Representation Theory (Coxeter Groups) **Field B** (complexity object): Complexity Theory: Boolean Function Entropy

Statement:

> ['For a boolean function f with n variables, the number of distinct simple transpositions that appear in the action of a Coxeter group on its associated Young diagram is upper bounded by $O(2^{n/\eta(f)}c)$, where η(f) is the nonlinearity of f and c is an absolute constant.', 'For all boolean functions f with n variables, if there exists a Coxeter group action that generates at least 2^n distinct simple transpositions on its associated Young diagram, then f has nonlinearity at least 2^α(n), where α is the inverse Ackermann function.']

Rationale (proposer's reasoning):

> ['Coxeter groups provide a combinatorial framework for studying symmetry in polynomials and other algebraic objects. Their actions can be used to analyze the complexity of boolean functions by considering the symmetries that are preserved or destroyed by the function.', 'The nonlinearity of a boolean function is a measure of its resistance to linear approximation, which is closely related to the complexity of computing the function. A connection between Coxeter group action complexity and boolean function entropy would provide new insights into both areas.']

Taxonomy category: `GCT_DET_PERM` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `bf575b3de8c8ee9a`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all boolean functions f with n variables, the number of distinct simple transpositions in the Coxeter group action on its associated Young diagram is within $O(2^{n/\eta(f)}c)$ of the expected value across 30 random seeds, and for those with a large number of transpositions, their nonlinearity meets or exceeds 2^α(n).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Coxeter group action" AND "Boolean function complexity" - "nonlinearity of Boolean functions" AND "Coxeter group diagrams" - "inverse Ackermann function" IN BOOLEAN FUNCTION papers

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_nonlinearity(f):
        n = int(math.log2(len(f)))
        max_linear_approximation_error = float('-inf')
        for j in range(n):
            linear_approximation = sum(f[i:i+n] for i in range(j, len(f), n))
            error = abs(linear_approximation - f[j])
            if error > max_linear_approximation_error:
                max_linear_approximation_error = error
        return 2**n - max_linear_approximation_error

    def compute_coxeter_group_action(f):
        n = int(math.log2(len(f)))
        transpositions = set()
        for i in range(n):
            for j in range(i+1, n):
                if f[i] != f[j]:
                    transpositions.add((i, j))
        return len(transpositions)

    def inverse_ackermann(n):
        a = [0, 1]
```

```
        for i in range(2, n + 1):
            a.append(a[-1] + 1)
        return a[n]

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        f = generate_boolean_function(n)
        nonlinearity = compute_nonlinearity(f)
        transpositions = compute_coxeter_group_action(f)

        if transpositions > 2**n / nonlinearity**2:
            counterexample = "Number of distinct simple transpositions exceeds O(2^n/η(f)^2)"
            return {
                "metric_name": "Transpositions",
                "metric_value": transpositions,
                "instances_tested": 1,
                "conjecture_holds": False,
                "counterexample": counterexample
            }

        if transpositions > 2**n / inverse_ackermann(n):
            counterexample = "Nonlinearity too low for number of distinct simple transpositions"
            return {
                "metric_name": "Transpositions",
                "metric_value": transpositions,
                "instances_tested": 1,
                "conjecture_holds": False,
                "counterexample": counterexample
            }

        results.append(transpositions)

    mean = sum(results) / len(results)
    std = math.sqrt(sum((x - mean)**2 for x in results) / len(results))
    support_fraction = len([r for r in results if r <= 2**n_values[-1] / inverse_ackermann(n_value

    return {
        "metric_name": "Transpositions",
        "metric_value": mean,
        "instances_tested": len(results),
        "conjecture_holds": support_fraction >= 0.8,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)
```

```
    mean = sum(r["metric_value"] for r in results) / len(results)
    std = math.sqrt(sum((r["metric_value"] - mean)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = "Number of distinct simple transpositions exceeds O(2^n/η(f)^c)"
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0c7547d6.py", line 97, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0c7547d6.py", line 54, in run_trial
    nonlinearity = compute_nonlinearity(f)
                   ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0c7547d6.py", line 28, in compute_nonli
    linear_approximation = sum(f[i:i+n] for i in range(j, len(f), n))
                           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: unsupported operand type(s) for +: 'int' and 'list'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed during execution, which prevents us from verifying the conjecture's conditions. | next: Investigate and fix the error in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12239
2	preregistration	ollama_remote	glm4:latest	0	0	5756
3	novelty	ollama_remote	glm4:latest	0	0	4561
4	novelty	ollama_remote	glm4:latest	0	0	5155
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14144
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11121
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10725
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11368
9	judge	ollama_remote	glm4:latest	0	0	11014

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 86082 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/1cd48f6fed7a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1cd48f6fed7a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/1cd48f6fed7a.tar.gz` (if generated)